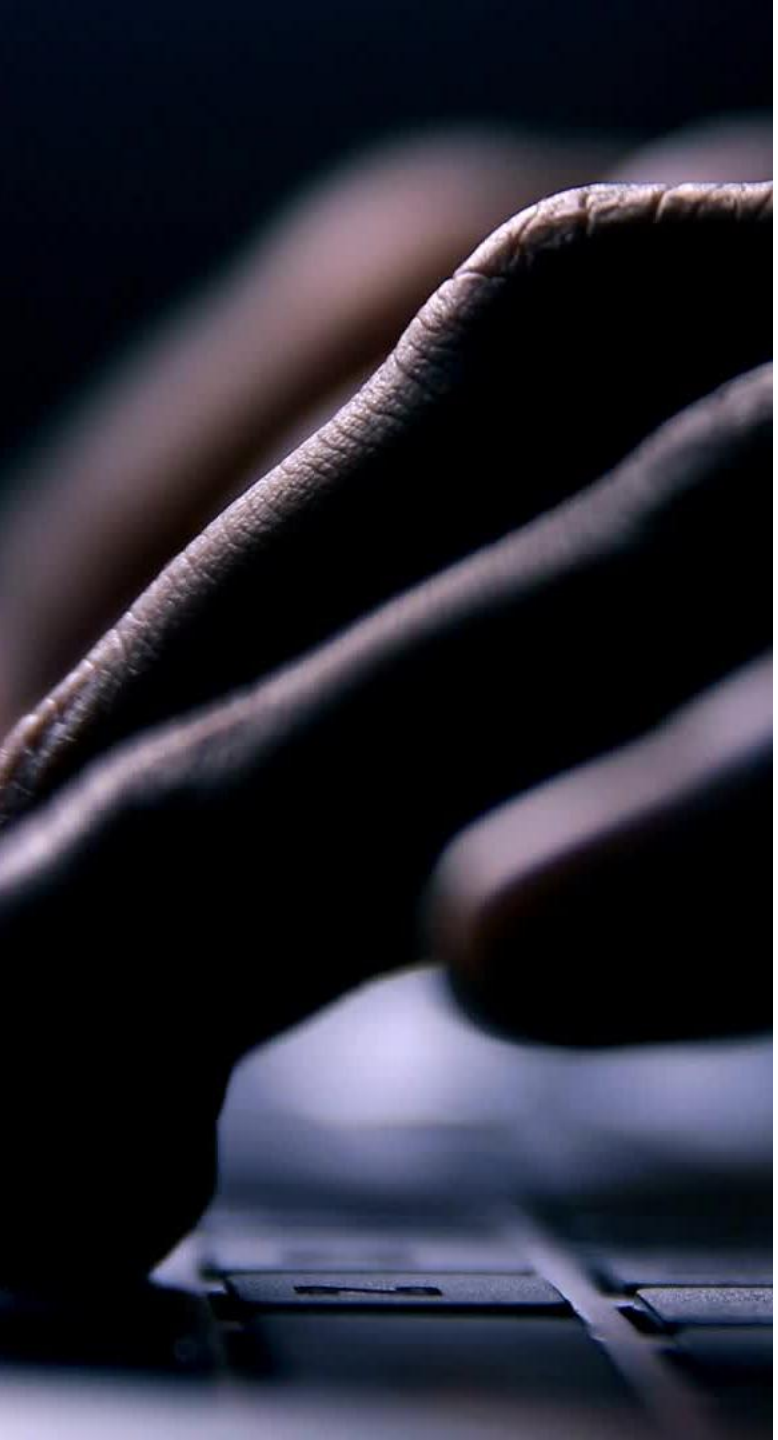


Fatec
Diadema
Luigi Papaiz

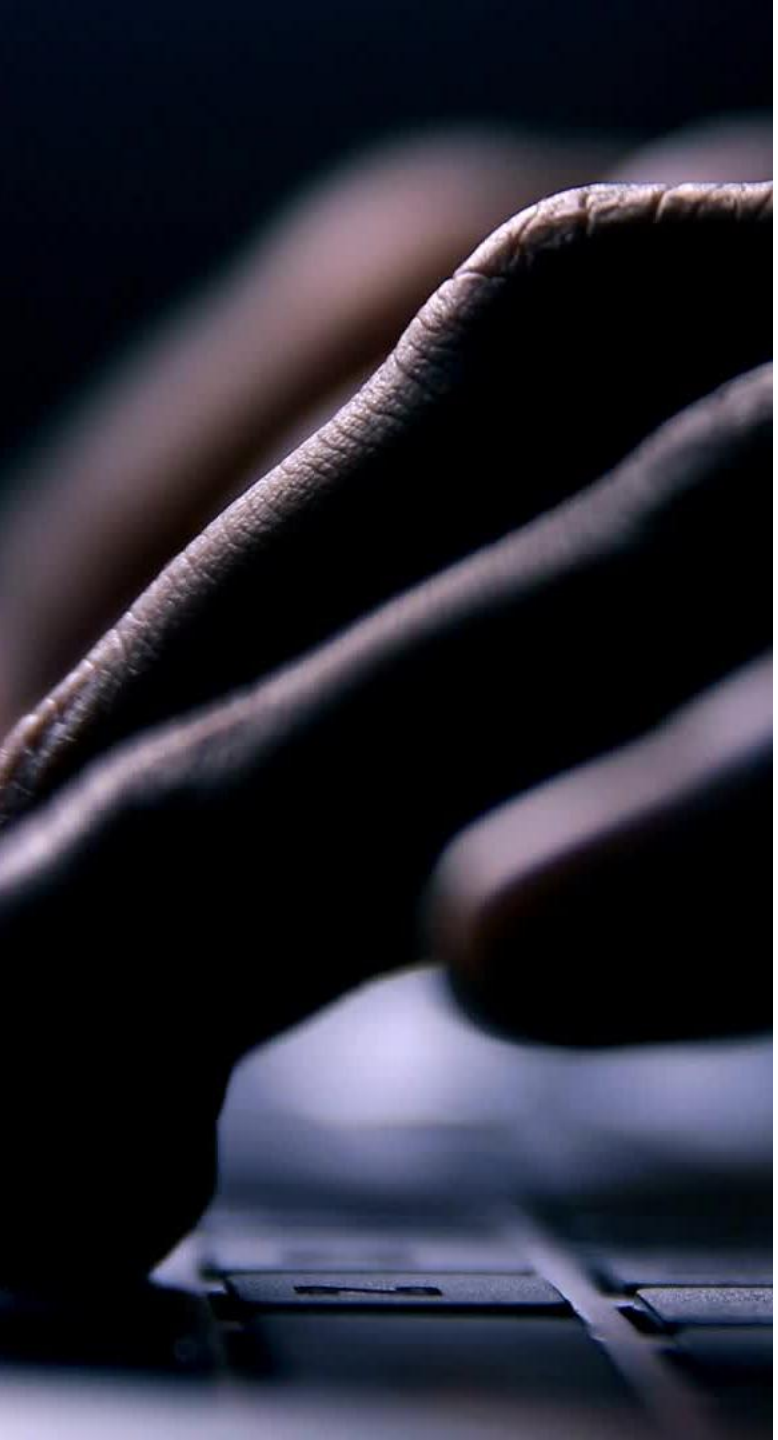
Programação para Dispositivos Móveis I

Prof. Rafael Miranda
rafael.miranda@cps.sp.gov.br

Na última aula

Fatec
Diadema
Luigi Papaiz

- Funções



Fatec
Diadema
Luigi Papaiz

Linguagem Kotlin - Coleções

Coleções

Coleções permitem armazenar e trabalhar com vários elementos.

Em Kotlin, podemos utilizar:

- Array
- List
- Set
- Map

O problema

Uma loja precisa desenvolver um sistema para controlar seus produtos.

Inicialmente, temos:

```
val produto1 = "Teclado"
val produto2 = "Mouse"
val produto3 = "Monitor"
```

E se a loja tiver centenas de produtos?

Array

Um **Array** é uma estrutura que armazena uma quantidade definida de elementos.

Os elementos são acessados por meio de **índices**.

```
val produtos = Array(3) { "" }

produtos[0] = "Teclado"
produtos[1] = "Mouse"
produtos[2] = "Monitor"
```

```
println(produtos[0])
```

Percorrendo um Array

Podemos percorrer todos os elementos:

```
for (produto in produtos) {
    println(produto)
}
```

Quantidade de elementos:

```
println(produtos.size)
```

Array na prática


```
fun main() {
    val produtos = arrayOf(
        "Teclado",
        "Mouse",
        "Monitor"
    )

    println(produtos[0])
    println(produtos.size)

    for (produto in produtos) {
        println(produto)
    }
}
```

List

Uma **List** representa uma coleção ordenada de elementos, que podem ser acessados por índice.

```
fun main() {
    val produtos = listOf(
        "Teclado",
        "Mouse",
        "Monitor"
    )

    println(produtos[0])
    println(produtos.size)
    println(produtos.contains("Mouse"))
    println(produtos.isEmpty())
}
```

Um novo problema

A loja começou com três produtos.

Mas agora precisamos:

- cadastrar novos produtos;
- remover produtos;
- manter a lista atualizada.

Precisamos de uma estrutura que permita alterar a quantidade de elementos.

MutableList

Uma MutableList é uma lista que permite adicionar, remover e alterar elementos.

```
fun main() {
    val produtos = mutableListOf(
        "Teclado",
        "Mouse",
        "Monitor"
    )

    produtos.add("Headset")
    produtos.remove("Mouse")
    produtos[0] = "Teclado Mecânico"

    println(produtos)
}
```

Mais um problema

A loja também precisa registrar as categorias dos produtos.

Uma mesma categoria pode aparecer várias vezes:

- Informática
- Eletrônicos
- Informática
- Acessórios
- Informática

Mas queremos armazenar cada categoria apenas uma vez.

Set

Um Set é uma coleção que não permite elementos duplicados.

```
fun main() {
    val categorias = setOf(
        "Informática",
        "Eletrônicos",
        "Informática",
        "Acessórios"
    )

    println(categorias)
    println(categorias.size)
}
```

A categoria "Informática" aparece apenas uma vez.

MutableSet

Um MutableSet permite alterar o conjunto.

```
fun main() {
    val categorias = mutableSetOf(
        "Informática",
        "Eletrônicos",
        "Acessórios"
    )

    categorias.add("Celulares")
    categorias.add("Informática")
    categorias.remove("Eletrônicos")

    println(categorias)
    println(categorias.size)

}
```

Outro problema

Agora precisamos consultar um produto pelo seu **código**.

Código	**Produto**
101	Teclado
102	Mouse
103	Monitor
104	Headset

Precisamos relacionar uma informação a outra.

Map

Um **Map** é uma coleção formada por pares de:

chave e valor

```
fun main() {
    val produtos = mapOf(
        101 to "Teclado",
        102 to "Mouse",
        103 to "Monitor",
        104 to "Headset"
    )

    println(produtos[102])
}
```

MutableMap

Um MutableMap permite adicionar, alterar e remover elementos.

```
fun main() {
    val produtos = mutableMapOf (
        101 to "Teclado",
        102 to "Mouse",
        103 to "Monitor",
        104 to "Headset"
    )

      produtos[105] = "Impressora"
      produtos[102] = "Mouse sem fio"
      produtos.remove(101)

      println(produtos)

}
```

Percorrendo um Map

Podemos percorrer as chaves e os valores:

```
for ((codigo, produto) in produtos) {
    println("$codigo - $produto")
}
```

List, Set e Map

List

Coleção ordenada, acessada por índice e que permite duplicados.

Set

Coleção que não permite elementos duplicados.

Map

Coleção baseada em **chave e valor**.

Funções e coleções

A loja precisa criar uma função para listar seus produtos.

```
fun listarProdutos(produtos: List<String>) {
    for (produto in produtos) {
        println(produto)
    }
}

fun main() {
    val produtos = listOf(
        "Teclado",
        "Mouse",
        "Monitor"
    )

    listarProdutos(produtos)
}
```

Uma coleção pode ser passada como parâmetro de uma função.

Uma nova forma de percorrer

Kotlin permite utilizar forEach para executar uma ação para cada elemento da coleção.

```
fun main() {
    val produtos = listOf(
        "Teclado",
        "Mouse",
        "Monitor"
    )

    produtos.forEach {
        println(it)
    }
}
```

it representa o elemento atual.

for x forEach

Podemos percorrer uma coleção utilizando:

Utilizando.

```
for (produto in produtos) {
    println(produto)
}
```

Ou

```
produtos.forEach {
    println(it)
}
```

Outro tipo de estrutura

Fatec
Diadema
Luigi Papaiz

E se precisarmos organizar os produtos em linhas e colunas?

A loja pode organizar seus produtos por:

Corredor × Posição

	Posição 0	Posição 1	Posição 2
Corredor 0	Teclado	Mouse	Monitor
Corredor 1	Headset	WebCam	Notebook
Corredor 2	Impressora	HD	SSD

Matriz

Uma **matriz** é uma estrutura organizada em linhas e colunas.

Cada elemento é acessado utilizando **dois índices**:

```
val produtos = Array(3) { Array(3) { "" } }


produtos[0][0] = "Teclado"
produtos[0][1] = "Mouse"
produtos[0][2] = "Monitor“
```

Percorrendo uma matriz


```
for (i in 0..2) {
    for (j in 0..2) {
        print("${produtos[i][j]} | ")
    }
    println()
}
```

Atividade: Sistema de notas

Fatec
Diadema
Luigi Papaiz

O professor precisa de um programa para organizar as notas dos alunos.

O programa deverá:

- cadastrar as notas dos alunos, associando cada aluno ao seu código;
- permitir consultar a nota de um aluno pelo código;
- permitir alterar uma nota cadastrada;
- apresentar todos os alunos e suas respectivas notas;
- informar a quantidade de alunos cadastrados;
- calcular a média da turma;
- informar a maior e a menor nota;
- apresentar os alunos que possuem nota igual ou superior a 7,0;
- apresentar também as diferentes notas que foram utilizadas na turma, sem repetir valores.